



Práctica Extra 1

Organización y Arquitectura de Computadoras

Docente: Garcia Rocha Jose Isabel

Alumno:

- Chaparro Herrera Hugo Giovanni - 2200357

Tijuana, Baja California a 30 de noviembre de 2025



Objetivo

Seleccionar las instrucciones correctas en aplicaciones de sistemas basados en microprocesador mediante la distinción de su funcionamiento, de forma lógica y responsable.

Desarrollo

Crear un programa en el cual el usuario ingrese una cadena de caracteres numericos terminados con el símbolo *, posteriormente se llame a la subrutina puts para mostrar la cadena en pantalla, luego se llame a la subrutina atoi para convertir la cadena a un número decimal, para finalmente llamar a la subrutina printf para mostrar el valor contenido en eax y posteriormente se muestra en pantalla con la subrutina printf.

Ejemplo:

Ingresa la cadena: 12345*

Salida en pantalla de puts: 12345

Salida en pantalla de printf: 00003039

Salida en pantalla de printf: 12345

El programa debe tener las siguientes subrutinas en un archivo .asm:

1. **gets:** Es una subrutina que captura en memoria direccionada por ESI, una cadena de caracteres y esta termina de almacenarlos hasta que se ingrese un símbolo '*', al final de la cadena almacenada debe colocar un 0.

void gets(int* direccion_donde_se_guarda_la_cadena_capturada);

Ejemplo: 'H'+'o'+'l'+'a'+' '+'m'+'u'+'n'+'d'+'o'+'*'

guardaría en memoria "Hola mundo", 0.

```
gets:
    push ebp                ; se guarda en la pila el valor de EBP
    mov ebp, esp            ; se manda ESP a EBP para poder extraer datos en base al puntero de pila

    pushad
    mov esi, [ebp + 8]      ; se manda la direccion de la cadena a ESI
    mov edi, esi            ; se copia el valor de ESI en EDI para manipularlo sin perder la referencia
ciclo_cap:
    ; sys_read
    mov eax, 3
    mov ebx, 0
    mov ecx, esi
    mov edx, 1
    int 80h

    cmp byte[esi], '*'      ; se compara con el indicador de fin de cadena
    je fingets              ; si al == *, se sale del ciclo capturar

    cmp byte[esi], 13        ; si al == ENTER, no se captura
```



```
je ciclo_cap

cmp byte[esi], 10 ; si al == NUEVA_LINEA, no se captura
je ciclo_cap

inc esi          ; se incrementa el indice
jmp ciclo_cap    ; se reinicia el ciclo

; imprimir cadena capturada
fingets:
mov byte[esi], 0 ; se coloca el caracter '0' en al
mov eax, edi     ; se coloca el puntero de la cadena en EAX para retornarla

popad
mov esp, ebp     ; se actualiza esp al nuevo puntero en la pila
pop ebp         ; se restaura el valor de EBP
ret
```

Primero por medio de un printf desde C, se le indica al usuario de ingresar una cadena de numeros, dicha cadena debe terminar con *. Esta cadena se va a leer usando gets.

```
// Se obtiene la cadena con GETS
printf("Ingrese una cadena de numeros (Terminada con *): ");
fflush(stdout);
gets((int*)cad);
```

Impresión del mensaje en terminal:

```
koria@GioLaptop:/mnt/c/Users/hugog/OneDrive/Documentos/6
Ingrese una cadena de numeros (Terminada con *): 12345*
```

Una vez capturada la cadena usando gets, se imprime un mensaje desde C que indica al usuario la cadena ingresada para posteriormente hacer uso de newputs e imprimir la cadena que ingresó el usuario hasta el último carácter antes del *

```
// Se imprime en pantalla la cadena de numeros con NEWPUTS
printf("\nSalida en pantalla de puts:    ");
fflush(stdout);
newputs(cad);
```

```
Salida en pantalla de puts:    12345
```



2. **atoi**: Es una subrutina que convierte una cadena de caracteres que representan un número decimal en un número decimal, recibe en ESI la dirección de la cadena y lo retorna en EAX, el máximo valor decimal a convertir es 4,294,967,295 o sea $(2^{32})-1$. Ejemplo “12345”, 0 es convertido a 12345 decimal. **unsigned int atoi(int* cadena_con_numero);**

```
atoi:
    push ebp          ; se guarda en la pila el valor de EBP
    mov ebp, esp      ; se manda ESP a EBP para la extracción de datos

    ; se guardan en la pila los registros a manipular
    push esi
    push ecx
    push ebx
    push edx
    mov esi, [ebp + 8] ; se manda a ESI la dirección de la cadena a convertir

    mov ecx, 0 ; se inicia en 0 para calcular la longitud
calc_len_atoi:
    cmp byte[esi + ecx], 0 ; se compara con el final de la cadena
    JE fin_len_atoi

    inc ecx ; se incrementa ecx
    jmp calc_len_atoi

fin_len_atoi:
    dec ecx          ; len - 1
    mov eax, 1        ; EAX = máxima base
    mov ebx, 10        ; Multiplicador/divisor

    ; se extrae la base del número en base a la longitud (ECX)
multiplicador:
    MUL ebx
    loop multiplicador
    mov ecx, eax      ; se manda a ECX la base máxima

    ; EAX se resetea a 0 para, aquí almacenaremos el número convertido para retornarlo al final
    mov eax, 0
convertir_atoi:
    cmp ecx, 0 ; se compara ECX con 0 para ver si se terminó de convertir
    je final

    ; se guarda EAX en la pila para iniciar el proceso de conversión de ASCII a NUM
    push eax
    mov eax, 0 ; se coloca en 0 EAX
    mov al, byte[esi] ; se extrae el carácter a convertir (MSB)
    sub al, '0'      ; se convierte a su valor numérico

    MUL ecx          ; se multiplica por la base
    mov edx, eax      ; se guarda en EDX el número convertido
```



```
pop eax

inc esi      ; nos desplazamos al siguiente caractera
add eax, edx ; se suma el numero a EAX (donde almacenamos valor del arreglo convertido a
numero)

; reducir a la siguiente base
push eax
push edx
mov edx, 0    ; se resetea a 0 EDX
mov eax, ecx  ; se manda a EAX la base maxima
div ebx      ; se divide entre 10 la base maxima
mov ecx, eax  ; se regresa a ECX la nueva base
pop edx
pop eax

jmp convertir_atoi
final:
pop edx
pop ebx
pop ecx
pop esi

mov esp, ebp ; se manda a ESP la nueva direccion a apuntar en la pila
pop ebp
ret
```

Una vez capturada la cadena de numeros, procedemos a hacer la conversion de dicha cadena. En C, declararemos una nueva variable del tipo **unsigned int** en la que guardaremos el valor convertido que retorna atoi.

```
// se convierte la cadena de caracteres a numero
unsigned int numero = atoi((int*)cad);
```



3. **printDec:** Es una subrutina que imprime los valores en formato decimal, ejemplo se ingresa en EAX un 64h muestra en pantalla un 100 decimal como cadena el número máximo mostrar debe ser de 4,294,967,295 o sea $(2^{32})-1$.

void printDec(unsigned int valor_a_imprimir);

```
printDec:
    push ebp                ; se guarda en la pila el valor de EBP
    mov ebp, esp            ; se manda ESP a EBP para la extracción de los datos
    pushad

    mov eax, [ebp + 8]      ; EAX contiene el número a convertir
    mov ebx, 0              ; EBX contiene la longitud
    mov esi, cad             ; ESI guarda un buffer auxiliar para la cadena
    mov ecx, 10             ; ECX guarda la base a convertir

convertir:
    mov edx, 0
    DIV ecx                 ; dividimos entre 10 para extraer

    add dl, '0'             ; convertimos a carácter
    push dx                 ; se manda a la pila el valor convertido
    inc ebx                 ; se incrementa EBX para sumar 1 a la longitud

    cmp eax, 0              ; se compara EAX para ver si la base es 0
    jne convertir           ; si no lo es, se continúa la conversión

; se extrae carácter por carácter para ser guardados en el buffer de impresión
guardar:
    pop dx                  ; se extrae el MSB
    mov byte[esi], dl       ; se guarda en la cadena para imprimirlo

    inc esi                 ; nos desplazamos a la siguiente posición
    dec ebx                 ; reducimos en 1 la longitud

    cmp ebx, 0              ; se compara EBX con 0 para ver si ya terminamos el guardado
    jne guardar             ; si no es así, continuamos

    mov byte[esi], 0        ; se coloca el carácter de fin de cadena
    mov esi, cad             ; se manda el buffer para hacer la impresión
imprimirDec:
    cmp byte[esi], 0        ; se compara para ver si ya terminamos la impresión
    je fin_printDec

; sys_call write
    mov eax, 4
    mov ebx, 1
    mov ecx, esi
    mov edx, 1
    int 80h
```



```
inc esi      ; nos desplazamos al siguiente caracter a convertir
jmp imprimirDec
fin_printDec:
popad
mov esp, ebp ; se actualiza ESP con la nueva direccion a apuntar en la pila
pop ebp
ret
```

Una vez convertido el número con atoi, usaremos dicho número y se lo mandaremos como parámetro a nuestras funciones printHex y printDec con la finalidad de imprimir en pantalla dicho número convertido en la base correspondiente, esto se lo indicamos al usuario por medio de printf.

```
// se imprime el numero con print-Hex
printf("\nSalida en pantalla de printHex: ");
fflush(stdout);
printHex(numero);

// se imprime el numero con print-Dec
printf("\nSalida en pantalla de printDec: ");
fflush(stdout);
printDec(numero);
```

```
koria@Giolaptop: /mnt/c/Users/hugog/OneDrive/Documentos/6to Semestre/OAC/Extra/PracticaExtra1$ ./P1
Ingrese una cadena de numeros (Terminada con *): 12345*

Salida en pantalla de puts:      12345
Salida en pantalla de printHex: 00003039
Salida en pantalla de printDec: 12345
```



Código C

```
#include <stdio.h>

// prototipos de Los procedimientos hechos en ASM
extern void gets(int* cad);           // GETS -> Captura cadena
extern int newputs(const char* cad);  // NEWPUTS -> Imprime cadenas
extern unsigned int atoi(int* cad);   // ATOI -> ASCII to INT
extern void printHex(unsigned int n); // PRINT-HEX
extern void printDec(unsigned int n); // PRINT-DEC

int main(void) {
    // Variable para que el usuario ingrese un numero a convertir
    char cad[50] = {0};

    // Se obtiene La cadena con GETS
    printf("Ingrese una cadena de numeros (Terminada con *): ");
    fflush(stdout);
    gets((int*)cad);

    // Se imprime en pantalla La cadena de numeros con NEWPUTS
    printf("\nSalida en pantalla de puts:   ");
    fflush(stdout);
    newputs(cad);

    // se convierte La cadena de caracteres a numero
    unsigned int numero = atoi((int*)cad);

    // se imprime el numero con print-Hex
    printf("\nSalida en pantalla de printHex: ");
    fflush(stdout);
    printHex(numero);

    // se imprime el numero con print-Dec
    printf("\nSalida en pantalla de printDec: ");
    fflush(stdout);
    printDec(numero);
    printf("\n");

    return 0;
}
```



Código en ASM

```
section .data
section .bss
    cad resb 8
section .text
    global gets:
    global newputs:
    global printDec
    global atoi
    global printHex:

; //////////////////////////////////////

gets:
    push ebp                ; se guarda en la pila el valor de EBP
    mov ebp, esp            ; se manda ESP a EBP para poder extraer datos en base al puntero de pila

    pushad
    mov esi, [ebp + 8]      ; se manda la direccion de la cadena a ESI
    mov edi, esi            ; se copia el valor de ESI en EDI para manipularlo sin perder la referencia
    ciclo_cap:
        ; sys_read
        mov eax, 3
        mov ebx, 0
        mov ecx, esi
        mov edx, 1
        int 80h

        cmp byte[esi], '*' ; se compara con el indicador de fin de cadena
        je fingets         ; si al == *, se sale del ciclo capturar

        cmp byte[esi], 13  ; si al == ENTER, no se captura
        je ciclo_cap

        cmp byte[esi], 10  ; si al == NUEVA_LINEA, no se captura
        je ciclo_cap

        inc esi            ; se incrementa el indice
        jmp ciclo_cap      ; se reinicia el ciclo

; imprimir cadena capturada
fingets:
    mov byte[esi], 0 ; se coloca el caracter '0' en al
    mov eax, edi      ; se coloca el puntero de la cadena en EAX para retornarla

    popad
    mov esp, ebp      ; se actualiza esp al nuevo puntero en la pila
    pop ebp           ; se restaura el valor de EBP
    ret
```



```
; ////////////////////////////////////////

newputs: ; recibe en EDI la direccion y en ESI el indice a mostrar
    push ebp                ; se guarda en la pila el valor de EBP
    mov ebp, esp            ; se manda ESP a EBP para poder manipular el puntero de pila
    pushad
    mov esi, [ebp + 8]      ; ESI apunta al inicio de la cadena pasada por pila
    mov edi, 0              ; se setea EDI en 0 para usarlo como contador de caracteres

ciclo_newputs2:
    cmp byte [esi + edi], 0 ; se compara el caracter actual con el indicador de caracter final
    je fin_ciclo_newputs2   ; si es asi, se sale del ciclo y procede a imprimir la cadena

    inc edi                  ; se le suma 1 al contador de caracteres
    jmp ciclo_newputs2

fin_ciclo_newputs2:
    ; SYS_CALL WRITE
    mov eax, 4
    mov ebx, 1
    mov ecx, esi
    mov edx, edi
    int 80h
    popad
    mov esp, ebp ; se actualiza a la nueva direccion apuntada
    pop ebp
    ret

; ////////////////////////////////////////

printHex:
    push ebp
    mov ebp, esp
    pushad

    mov eax, [ebp + 8]
    mov esi, cad
    mov edx, eax
    mov ebx, 0fh
    mov cl, 28
.nxt: shr eax, cl
.msk: and eax, ebx
    cmp al, 9
    jbe .menor
    add al, 7
.menor: add al, '0'
    mov byte [esi], al
    inc esi
    mov eax, edx
```



```
    cmp cl, 0
    je .print
    sub cl, 4
    cmp cl, 0
    ja .nxt
    je .msk
.print:
    mov byte[esi], 0
    mov eax, 4
    mov ebx, 1
    sub esi, 8
    mov ecx, cad
    mov edx, 8
    int 80h

    popad
    mov esp, ebp
    pop ebp
    ret

; //////////////////////////////////////

atoi:
    push ebp          ; se guarda en la pila el valor de EBP
    mov ebp, esp      ; se manda ESP a EBP para la extracción de datos

    ; se guardan en la pila los registros a manipular
    push esi
    push ecx
    push ebx
    push edx
    mov esi, [ebp + 8] ; se manda a ESI la dirección de la cadena a convertir

    mov ecx, 0 ; se inicia en 0 para calcular la longitud
calc_len_atoi:
    cmp byte[esi + ecx], 0 ; se compara con el final de la cadena
    JE fin_len_atoi

    inc ecx ; se incrementa ecx
    jmp calc_len_atoi

fin_len_atoi:
    dec ecx          ; len - 1
    mov eax, 1        ; EAX = máxima base
    mov ebx, 10        ; Multiplicador/divisor

    ; se extrae la base del número en base a la longitud (ECX)
multiplicador:
    MUL ebx
    loop multiplicador
```



```
mov ecx, eax    ; se manda a ECX la base maxima

; EAX se resetea a 0 para, aqui almacenaremos el numero convertido para retornarlo al final
mov eax, 0
convertir_atoi:
    cmp ecx, 0 ; se compara ECX con 0 para ver si se termino de convertir
    je final

    ; se guarda EAX en la pila para iniciar el proceso de conversion de ASCII a NUM
    push eax
    mov eax, 0 ; se coloca en 0 EAX
    mov al, byte[esi] ; se extrae el caracter a convertir (MSB)
    sub al, '0'      ; se convierte a su valor numerico

    MUL ecx          ; se multiplica por la base
    mov edx, eax     ; se guarda en EDX el numero convertido
    pop eax

    inc esi          ; nos desplazamos al siguiente caractera
    add eax, edx     ; se suma el numero a EAX (donde almacenamos valor del arreglo convertido a
numero)

    ; reducir a la siguiente base
    push eax
    push edx
    mov edx, 0      ; se resetea a 0 EDX
    mov eax, ecx    ; se manda a EAX la base maxima
    div ebx         ; se divide entre 10 la base maxima
    mov ecx, eax    ; se regresa a ECX la nueva base
    pop edx
    pop eax

    jmp convertir_atoi

final:
    pop edx
    pop ebx
    pop ecx
    pop esi

    mov esp, ebp    ; se manda a ESP la nueva direccion a apuntar en la pila
    pop ebp
    ret

; //////////////////////////////////////

printDec:
    push ebp        ; se guarda en la pila el valor de EBP
    mov ebp, esp    ; se manda ESP a EBP para la extraccion de los datos
    pushad
```



```
mov eax, [ebp + 8]    ; EAX contiene el numero a convertir
mov ebx, 0            ; EBX contiene la longitud
mov esi, cad          ; ESI guarda un buffer auxiliar para la cadena
mov ecx, 10           ; ECX guarda la base a convertir

convertir:
    mov edx, 0
    DIV ecx           ; dividimos entre 10 para extraer

    add dl, '0'       ; convertimos a caracter
    push dx           ; se manda a la pila el valor convertido
    inc ebx           ; se incrementa EBX para sumar 1 a la longitud

    cmp eax, 0        ; se compara EAX para ver si la base es 0
    jne convertir     ; si no lo es, se continua la conversion

; se extrae caracter por caracter para ser guardados en el buffer de impresion
guardar:
    pop dx            ; se extrae el MSB
    mov byte[esi], dl ; se guarda en la cadena para imprimirlo

    inc esi           ; nos desplazamos a la siguiente posicion
    dec ebx           ; reducimos en 1 la longitud

    cmp ebx, 0        ; se compara EBX con 0 para ver si ya terminamos el guardado
    jne guardar       ; si no es asi, continuamos

mov byte[esi], 0      ; se coloca el caracter de fin de cadena
mov esi, cad          ; se manda el buffer para hacer la impresion
imprimirDec:
    cmp byte[esi], 0 ; se compara para ver si ya terminamos la impresion
    je fin_printDec

    ; sys_call write
    mov eax, 4
    mov ebx, 1
    mov ecx, esi
    mov edx, 1
    int 80h

    inc esi           ; nos desplazamos al siguiente caracter a convertir
    jmp imprimirDec

fin_printDec:
    popad
    mov esp, ebp      ; se actualiza ESP con la nueva direccion a apuntar en la pila
    pop ebp
    ret
```



Dificultades durante el desarrollo

Durante el desarrollo de la práctica, las mayores dificultades presentadas fueron la vinculación entre C y ASM, ya que, hay aspectos del pensamiento crítico en los que se debe uno adaptar, por ejemplo, cuando se vinculan estos dos lenguajes de programación, el principal cambio es en la manera de cómo se manipulan los datos, ya que estos datos principalmente se extraen desde la pila. Por lo que, esto fue un reto al comienzo, sin embargo, una vez adaptándose, puedes programar sin problemas.

Referencias Bibliográficas

- [02. NASMx86: partes del código fuente](#)
- [Guide to x86 Assembly](#)
- [Using as - Assembler Directives](#)
- [NASM Assembly Language Tutorials - asmtutor.com](#)
- [Assembly - System Calls](#)